MANUAL

- What it does
- Before you start
- Connect
- Tools
- Test scenarios
- Errors
- Known limits
- Sources

RADAR IMMOBILIER · PREPROD · USER MANUAL

Testing the Radar immobilier MCP server in preprod

Connect Claude to the preprod MCP endpoint, call its 16 tools, publish an HTML artifact and read the typed errors.

MCP endpoint	`https://preprod.immo.sent-tech.ca/mcp`
Identity provider	`https://preprod.auth.sent-tech.ca`
Artifact hosts	`<artifact-uuid>.preprod.immo-pages.sent-tech.ca` (stable) and `<release-uuid>.pinned.preprod.immo-pages.sent-tech.ca` (pinned)
Served API release	`6092169`, from `GET /health` on 2026-10-30
Source read	`origin/main` at `d25eb5ac2`, which contains `6092169`
Repository copy	`docs/manuals/MCP_PREPROD_USER_MANUAL.md`, branch `docs/mcp-preprod-manual`

What the MCP server does

The server lets Claude read Radar immobilier data during a conversation: regulatory signals, zoning and cadastral GeoJSON, zoning lifecycle events and PDF links.

It also lets Claude publish small HTML pages (artifacts) on the `immo-pages` hosts, readable through the Radar application login.

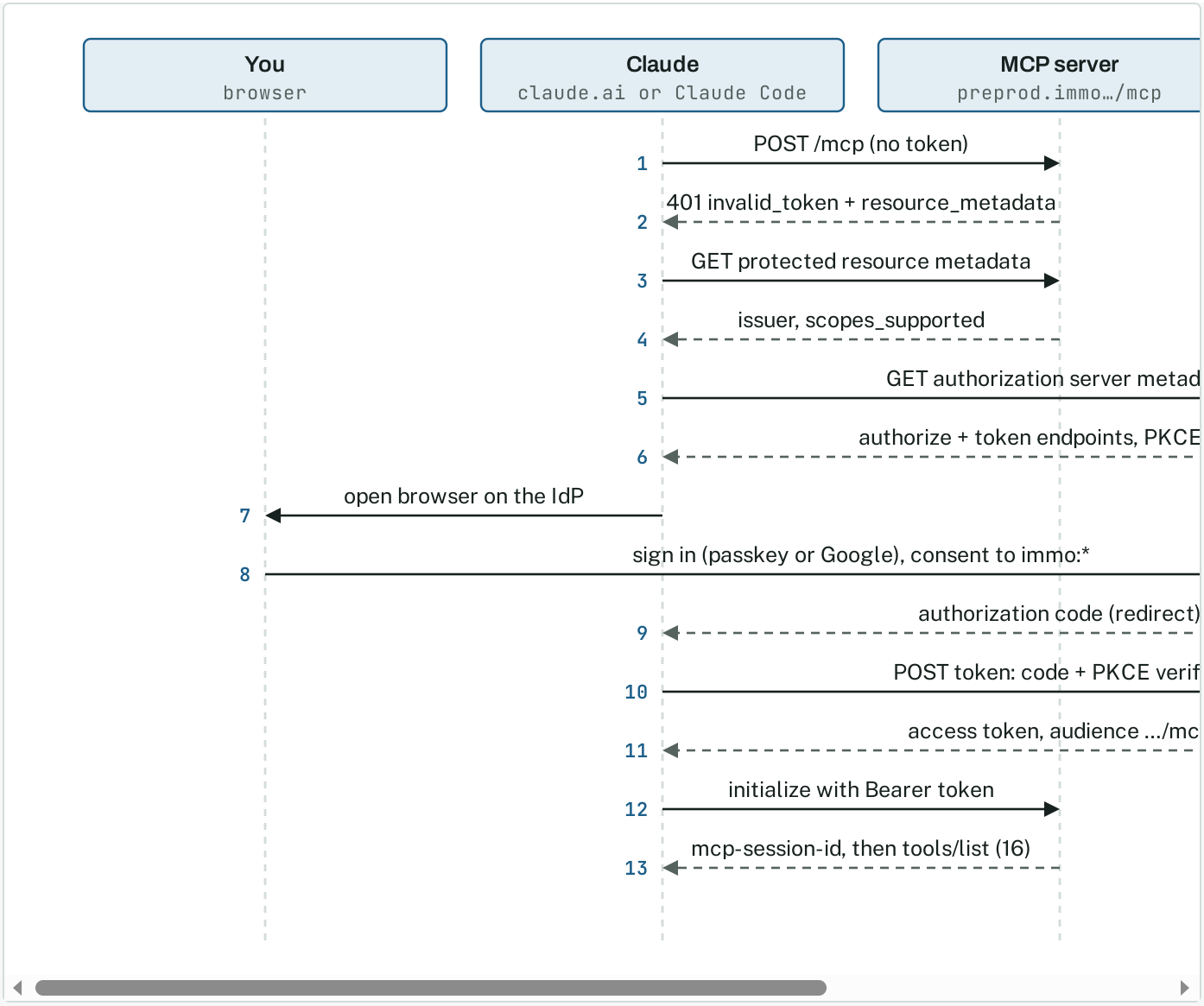
Every call runs under your own identity. The server never takes identity, workspace or ownership from tool arguments.

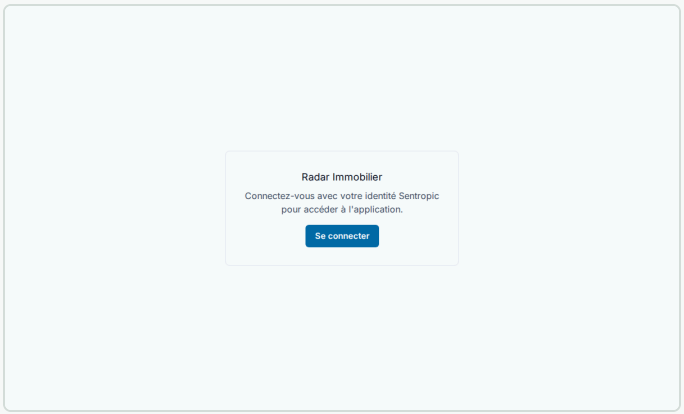
Before you start

- **Approved account.** Your Sentropic identity must have an `approved` account in the preprod application. The server rereads this status on every call.
- **One active workspace** for `search_signals`. The tool has no workspace selector: zero workspaces fails with `workspace_missing`, several with `workspace_ambiguous`. Publishing artifacts needs no workspace.
- **Scopes.** The `/mcp` route requires `immo:read`. `search_signals` and `search_lots` also need `immo:search`; document tools need `immo:documents:read`. Grant all three on the consent screen. With `immo:read` only, artifacts and GeoJSON work but `search_signals` returns `scope_denied:immo:search`.
- **OAuth client ID.** The preprod IdP metadata advertises no `registration_endpoint` (checked 2026-10-30), so clients cannot register themselves. You need a pre-registered public client ID. The preprod value is not recorded in the repository (`missing source`): ask the IdP owner. Production uses the public client `immo-mcp` with no secret.

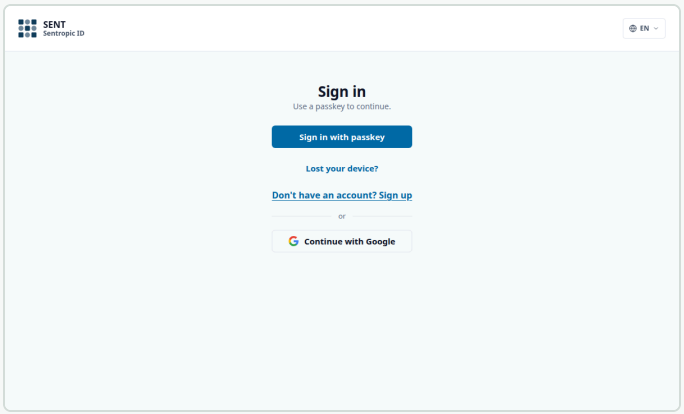
Connect

OAuth flow





The application, signed out. Only a sign-in card is public.



The IdP sign-in screen. Sign in with a passkey or Google; nothing is typed into Claude.

```
$ curl -i -X POST https://preprod.immo.sent-tech.ca/mcp  (no Authorization header) — live response, 2026-10-30

HTTP/2 401
content-type: application/json
date: Fri, 30 Oct 2026 21:38:16 GMT
www-authenticate: Bearer error="invalid_token", error_description="Authorization header is required.", resource_metadata="http
s://preprod.immo.sent-tech.ca/.well-known/oauth-protected-resource/mcp"
content-length: 80

{"error":{"code":"invalid_token","message":"Authorization header is required."}}
```

Live `curl -i` of `/mcp` without a token, 2026-10-30. The 401 and its `resource_metadata` pointer start the flow; they are expected.

Pretty-print


```
{"resource":"https://preprod.immo.sent-tech.ca/mcp","authorization_servers":["https://preprod.auth.sent-tech.ca"],"bearer_methods_supported":
["header"],"dpop_signing_alg_values_supported":["EdDSA"],"scopes_supported":["immo:read","immo:search","immo:documents:read"]}
```


The discovery document the client reads next: issuer and supported scopes.

claude.ai (custom connector)

Pro, Max, Team or Enterprise plan. On Team and Enterprise an admin may have to add it.

1. Open **Customize** → **Connectors** (`https://claude.ai/customize/connectors`).
2. Click **+**, then **Add custom connector**.
3. Name: `Radar immobilier (preprod)` . Remote MCP server URL: `https://preprod.immo.sent-tech.ca/mcp` .
4. In **Advanced settings**, set **OAuth Client ID** to the preprod client ID. Leave **OAuth Client Secret** empty.
5. Click **Add**, open the connector, click **Connect**.
6. Sign in on the IdP screen and accept the scopes.
7. The connector shows as connected with its tools. Enable it in each new conversation under **+** → **Connectors**.

Claude Code

```
claude mcp add --transport http \
  --client-id <PREPROD_CLIENT_ID> \
  --callback-port <PORT> \
  -s project immo-preprod https://preprod.immo.sent-tech.ca/mcp
```

Then run `/mcp` , select `immo-preprod` and choose **Authenticate**. A browser opens on the IdP sign-in screen. `--callback-port` fixes the local redirect port; whether the preprod client needs a fixed loopback redirect is `unverified` .

With `-s project`, Claude Code 2.1.314 writes this entry to `.mcp.json`:

```
{
  "mcpServers": {
    "immo-preprod": {
      "type": "http",
      "url": "https://preprod.immo.sent-tech.ca/mcp",
      "oauth": { "clientId": "<PREPROD_CLIENT_ID>", "callbackPort": 53682 }
    }
  }
}
```

The repository's own `.mcp.json` declares `immo` as a local stdio server on synthetic fixtures with a stub identity. It never reaches preprod.

Tools

With artifacts in `general` mode, `tools/list` returns 16 tools: 12 Radar tools (read-only) and 4 artifact tools. Results are JSON text; failures set `isError: true` with an `error` code. City values are slugs such as `longueuil`, `delson`, `valleyfield`.

TOOL	SCOPE	PREPROD
`search_signals`	`immo:search`	live scoped API facade
`search_lots`	`immo:search`	not_wired_yet
`get_lot_card`	`immo:read`	not_wired_yet
`get_opportunity_dossier`	`immo:read`	not_wired_yet
`list_documents`	`immo:documents:read`	not_wired_yet
`read_document_excerpt`	`immo:documents:read`	not_wired_yet
`get_zones_geojson`	`immo:read`	expected live, unverified
`get_lots_geojson`	`immo:read`	expected live, unverified
`get_grille_pdf`	`immo:documents:read`	expected live, unverified
`get_pv_pdf`	`immo:documents:read`	unverified
`get_reglement_provenance`	`immo:read`	expected live, unverified

TOOL	SCOPE	PREPROD
`query_zoning_events`	`immo:read`	expected live, unverified
`artifacts_ensure`	`immo:read`	live general mode
`artifacts_publish`	`immo:read`	live general mode
`artifacts_list`	`immo:read`	live general mode
`artifacts_get`	`immo:read`	live general mode

Example results come from the repository fixture `immo-tools.v1.json` (synthetic data) or from the code. They show the shape, not preprod data.

`search_signals`

Regulatory signals for a city. Parameters: `city` (required), `etape` (`avis_motion` , `adoption` , `consultation` ...), `query` (keyword), `showDiscarded` (default false), `limit` (1–100, default 20).

```
{ "name": "search_signals", "arguments": { "city": "longueuil", "etape": "avis_motion
```

```
{ "city": "longueuil", "count": 1,
  "signals": [{ "id": "sig-longueuil-001", "city": "longueuil", "type": "modification
    "etape": "avis_motion", "regulatoryStatus": "anticipation", "etape_date": "2026-0
    "reglement_number": "CO-2026-1187", "zone_ref": "H-203", "no_lot": "6 359 591",
    "summary": "Avis de motion pour permettre l'habitation multifamiliale jusqu'à 4 é
    "source_document_id": "doc-longueuil-pv-2026-04-14" }] }
```

`search_lots` , `get_lot_card` , `get_opportunity_dossier` , `list_documents` , `read_document_excerpt`

Parameters: `search_lots` takes `city` , `zone` , `no_lot` , `minArea` (m^2), `limit` . `get_lot_card` takes `city` and `no_lot` . `get_opportunity_dossier` takes `city` and `opportunityId` . `list_documents` takes `city` , `limit` . `read_document_excerpt` takes `documentId` , `offset` , `maxChars` (1–4000, default 1200). In preprod all five answer:

```
{ "error": "not_wired_yet:http_data_source GET /api/geo/:city/lots?no_lot= base=… (on
  "tool": "get_lot_card", "correlationId": "…" }
```

`get_zones_geojson` and `get_lots_geojson`

GeoJSON FeatureCollections of zones or cadastral lots. Parameters: `city` (required), `bbox` `[minLon, minLat, maxLon, maxLat]` (required when `limit` > 500), `limit` (1–2000, default 500). Lots add `only4Plus` and `superficieMinM2`, and carry `zoneCode`, the joined `zone`, `multifamilial4plus` and `superficieM2`.

```
{ "name": "get_lots_geojson", "arguments": { "city": "delson", "only4Plus": true } }
```

```
{ "city": "delson", "collectionId": "qc-lots-delson",
  "featureCollection": { "type": "FeatureCollection", "features": [ … ] },
  "numberMatched": 53, "numberReturned": 53, "truncated": false }
```

If `truncated` is true, narrow the bbox. The 53 lots figure comes from the production connector guide.

`get_grille_pdf`, `get_pv_pdf`

`get_grille_pdf` (`city`, `zone`) returns the public zoning grid URL: `{ "found": true, "zone": "H-203", "grillePdfUrl": "https://…/H-203.pdf" }`. `get_pv_pdf` (`rawRef`, such as `raw/proces-verbaux-longueuil/cas/<sha>.pdf`) returns the served URL of the source minutes; the PDF is never inlined. Unknown: `{ "found": false, "note": "statut api: 404" }`.

`get_reglement_provenance`, `query_zoning_events`

`get_reglement_provenance` (`muni_slug`; `zone_code` accepted, no effect) returns the by-law in force: `reglement_numero`, `reglement_millesime`, `reglement_page_source`, `reglement_url`. Missing fields are `null`. `query_zoning_events` (`muni_slug`, `zone_code`, `document_type`, `decision_state` = `planned|decided`) returns `{ city, count, events[], numberMatched, truncated }`. A `planned` event is never an adoption.

`artifacts_ensure`

Creates or finds your artifact for a binding name. Parameters: `bindingName` (1–128, required), `idempotencyKey` (1–256, required), `parameters` (empty object).

```
{ "name": "artifacts_ensure", "arguments": { "bindingName": "manual-test", "idempoten
→ { "artifactId": "0b5c7f1e-…", "created": true }
```

`artifacts_publish`

Publishes a release of an artifact you own. Parameters: `artifactId` (lowercase UUID), `idempotencyKey`, `files` (1–32 entries of `{ path, mimeType, text }` or `bytes`), optional flat `profile`. Limits: 256 KiB decoded in total; MIME types `text/html`, `text/css`, `text/javascript`, `application/json`; no path starting with `api`, `__artifacts`, `livez` or `health`. `index.html` is served at `/`. Fields such as `owner` or `tenant` are refused.

```
{ "name": "artifacts_publish", "arguments": {
  "artifactId": "0b5c7f1e-…", "idempotencyKey": "manual-test-publish-1",
  "files": [{ "path": "index.html", "mimeType": "text/html",
              "text": "<!doctype html><title>MCP test</title><h1>Hello from preprod</
→ { "artifactId": "0b5c7f1e-…", "releaseId": "7d2a…", "manifest": "…", "digest": "…",
    "stableUrl": "https://0b5c7f1e-….preprod.immo-pages.sent-tech.ca/",
    "pinnedUrl": "https://7d2a….pinned.preprod.immo-pages.sent-tech.ca/" }
```

`artifacts_list`, `artifacts_get`

`artifacts_list` (`limit` 1–100 default 20, `offset`) returns `{ total, artifacts: [{ artifactId, ownerId, state, isPublic }] }`. `artifacts_get` (`artifactId`) returns `{ artifactId, ownerId, storageScopeId, state, isPublic, withdrawn, currentReleaseId, grants }`; `grants` appears only for the owner.

Test scenarios

(a) Find the signals of a city

1. With the connector enabled, ask: *“With Radar immobilier preprod, list the notices of motion in Longueuil.”*
2. Claude calls `search_signals` with `{ "city": "longueuil", "etape": "avis_motion" }`. Accept the tool permission prompt.

> EXPECTED

`{ city, count, signals }` where every signal has `city = "longueuil"` and `etape = "avis_motion"`, and `count` equals the list length. The signals match what the application shows for the same account and city. `count: 0` is valid when the workspace has none. `scope_denied:immo:search` means you must reconnect and grant `immo:search`.

(b) Open a lot card or an opportunity dossier

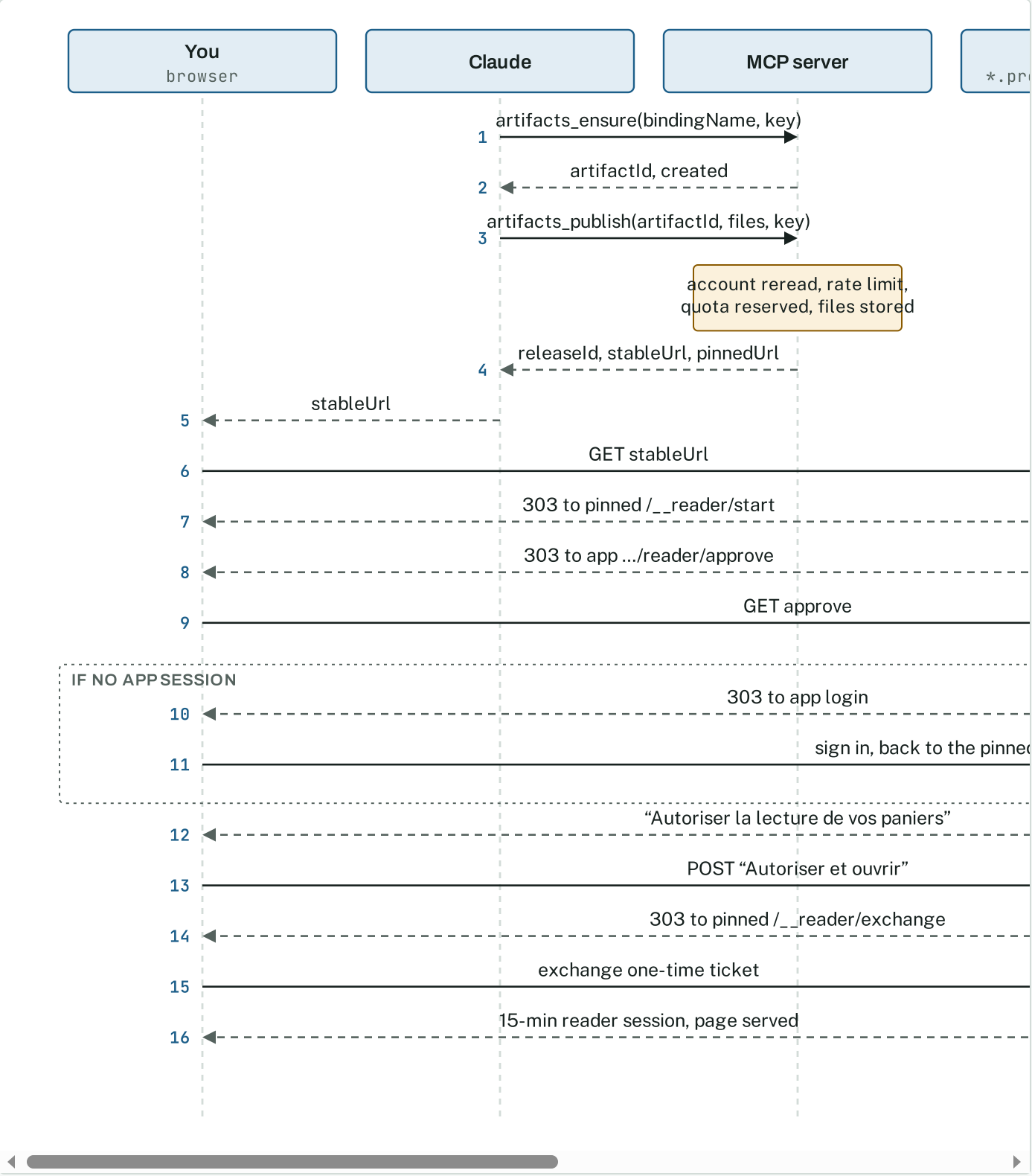
1. Ask: *"Open the lot card of lot 6 359 591 in Longueuil."*
2. Claude calls `get_lot_card` with `{ "city": "longueuil", "no_lot": "6 359 591" }`.

EXPECTED

`isError: true` with `not_wired_yet:http_data_source GET /api/geo/:city/lots?no_lot=…`. `get_opportunity_dossier` answers the same way. This is the current contract.

Working alternative: *"Get the lots of Delson with only4Plus and show lot 3 129 958 with its zone."* Claude calls `get_lots_geojson` and reads `zoneCode`, `zone.grillePdfUrl`, `superficieM2`; then `get_grille_pdf` gives the grid PDF.

(c) Publish an HTML artifact and open it



1. Ask: *“Using artifacts_ensure with bindingName manual-test, then artifacts_publish, publish a one-page index.html that says Hello from preprod, and give me the stable URL.”*
2. Claude calls `artifacts_ensure`, then `artifacts_publish`.
3. Open the `stableUrl` in a browser where you can sign in to the Radar application. Sign in if asked.

4. The application shows **“Autoriser la lecture de vos paniers”** with page ID, author and version. Click **Autoriser et ouvrir**.

> EXPECTED
>
> - `artifacts_publish` returns `stableUrl` and `pinnedUrl` on `preprod.immo-pages.sent-tech.ca`.
> - The browser ends on the pinned host and renders your HTML. The reader session lasts 15 minutes.
> - `artifacts_list` includes the artifact; `artifacts_get` shows `currentReleaseId` equal to the returned `releaseId`.
> - The same `idempotencyKey` with the same files returns the same release without a new charge; with different files it returns `conflict`.



A URL for an artifact that does not exist answers `not_found` and never shows the application.

Errors

HTTP errors happen before any tool runs and are usually handled by the client. Tool errors come back as a result with `isError: true` and a JSON `error` code. Search errors also carry `retryable`, `action` and a `correlationId`: give the `correlationId` to the operator when you report a problem.

SITUATION	WHAT YOU GET	WHAT TO DO
401: no or expired token	HTTP 401, `WWW-Authenticate: Bearer error="invalid_token"`	Reconnect: claude.ai **Connect**, or `/mcp` → Authenticate in Claude Code
Token lacks `immo:read`	HTTP 401 `invalid_token` or 403 `insufficient_scope` depending on the `mcp-auth` revision (`unverified`)	Reconnect and accept all scopes
Tool scope missing	`scope_denied:immo:search` or `:immo:documents:read`	Reconnect and grant the scope

SITUATION	WHAT YOU GET	WHAT TO DO
403, account suspended or not approved	`composition_account` (checked before every call); `account_not_approved` (403) from the search facade or artifact admission	Ask the operator to restore approval. Your token stays valid but every call is refused
Workspace: not a member	`workspace_missing` (403), `workspace_not_authorized` or `workspace_membership_required`; `composition_account` if your token asserts a workspace you left	Ask the operator to restore the membership. Artifacts still work in general mode
Several workspaces	`workspace_ambiguous` (409)	Operator configuration; do not retry
503 capability_unavailable, search	`signal_capability_unavailable`, `upstreamStatus: 503`, `action: contact_operator`	Search facade closed: contact the operator
503 capability_unavailable, artifacts	`{ "error": "capability_unavailable", "retryable": true }`	Retry after a minute with the same `idempotencyKey`. `retryable: false` means not configured: contact the operator
Server composition down	HTTP 503 `composition_unavailable`	Contact the operator
Account database unreachable	`account_binding_unavailable`, `retriable: true`	Retry later
Quota: rate	`rate_limited` (429, retryable), more than 10 artifact writes per minute	Wait a minute, retry with the same key
Quota: ceiling	`conflict` (409, not retryable): your operation or byte quota, or the preprod scope quota, is used up	Ask the operator to raise the quota
Same key, other content	`conflict` (409)	Use a new `idempotencyKey`
Bad arguments	`invalid_request`	Fix the arguments
Not your artifact	`access_denied` (403)	Use `artifacts_ensure` for your own artifact
Session expired	JSON-RPC 404 “Unknown or expired MCP session” (30 min idle, 8 h total)	The client opens a new session; reconnect if not

Known limits

Two-account acceptance not done. The artifacts runbook requires two approved accounts in two browser profiles: distinct artifacts, no cross-account access, shared ACL, cursor continuation across an API restart, host isolation checks. Not run; cross-account isolation is `unverified` in preprod.

Strict receiver disarmed. Search runs with `STRICT_RECEIVER_ARMED=0` (runbook 6H step 7). Steps 8–10 (A2 receipt, arming, suspended-account and revoked-membership probes) are not recorded, so the API still accepts the plain human bearer path and the activation stays `unverified`. Test with designated test accounts only.

- Five domain tools return `not_wired_yet`.
- The preprod OAuth client ID for connectors is not documented in the repository.
- No authenticated MCP call was made while writing this manual; results shown are shapes from fixtures and code.
- Mutations other than artifacts (notes, decisions) are not exposed.

Sources

- `packages/immo-mcp/src/` : `tool-definitions.ts`, `server-http.ts`, `data-source.ts`, `signal-read-error.ts`, `account-binding.ts`, `artifact-binding.ts`, `registry/artifact-profile.ts`, `registry/artifact-provider.ts`, `registry/stateful-composition.ts`
- `packages/artifacts-host/src/` (admission, quotas, rate limit) and `@sentropic/artifacts@0.2.2` (schemas, failure codes)
- `api/src/artifacts/reader-opening.ts` (reader approval flow)
- `docs/runbooks/ARTIFACTS_PREPROD_ACTIVATION.md`, `docs/runbooks/uimcp-6h-search-signals-activation.md`, `RUNBOOK-PREPROD-MCP-RACCORDEMENTS.md`, `docs/spec/SPEC_BASE_API.md`, `.mcp.json`
- Screenshots: headless Chromium (local Playwright), 2026-10-30, public pages only, no token or secret on screen.